

Hello!

I am Sayeed Unnisa

I am here because I love to give presentations on **Java Full Stack**

You can find me at sayeedunnisa.js@gmail.com

SPRING DATA JPA

ORM: Object Relational Mapping

- JDBC API given by Java, used to perform database Operations.

Employee object => eob = (eid=10,ename="A",esal=300.0)

=> SQL : insert into emptab values(eob.eid, eob.ename, eob.esal);

- ORM says perform Database operations using Objects Concept.(OOPs)

[save(obj), update(obj), delete(obj), get(id):obj ..etc]

Mapping Rule

1 class = 1 Table

1 variable = 1 Column

1 Object = 1 Row

- Programmer has to map a Java class (Entity) with one Database table
- Objects are converted into Rows (Rows-> Objects)
- JPA (Java Persistency API) is given by Sun/Oracle to perform Database Operations using ORM concept.
- For JPA API, there are vendors like Hibernate, iBatis, Eclipse Link,..etc

Properties

1. dialect:

It is a class, that generates SQL query when we perform operation

Ex: Oracle10gDialect, MySQL8Dialect, ..etc

All these dialects are present in package: org.hibernate.dialect

2. show_sql : true

To display Generated SQL at console, we have to set this property as true

3. hbm2ddl.auto [Hibernate Mapping to DDL Queries]

- **create**
- **update**
- **validate**
- **create-drop**

JPA: Java Persistency API - Sun/Oracle (Specification)[1st party]

[Set of Rules, interfaces, class, enum, Annotation]

Hibernate, iBatis, Eclipse link ...etc (Implementations) [Vendors/3rd parties]

We use JPA with one Vendor and write application to perform
database operations

Spring Boot - Data JPA

JPA : Specification

Hibernate : Vendor/Implementation

➤ To Perform Database Operation using ORM, we need JPA with Hibernate.

➤ Here we write 3 files:

1. **Entity class**
2. **Persistence xml file**
3. **Operations class**

- Operations code is a boiler-plate code [a type of common/duplicate code]
- Such code is generated at runtime based on Entity class and PrimaryKey inputs, using one design pattern dynamic proxy class.
- A class is generated using set of interfaces common instructions (or common type of code) with one object.
- In JPA with Hibernate, programmer has to write code for Operations.
- If we use Spring Boot - Data JPA then Operations code is generated.

Data JPA also given code for

1. CRUD Operations
2. Pagination
3. Sort
4. Custom Query (JPQL/HQL)

Delete Query

Update Query

JOINS Select

➤ If we use Spring Boot - Data JPA, we need to define 2 main files for coding, those are:

1. Entity class (same as JPA)
2. Repository interface (A class is generated at runtime)
3. Test Runner class : To check the output

➤ Define one Repository interface, by extending any one of below interfaces:

1. CrudRepository<T, ID>
2. PagingAndSortingRepository<T, ID>
3. JpaRepository<T, ID>

- Spring Data JPA reduces operations code
- It has given pre-defined Repositories to perform operations
- One Proxy class is generated at runtime.

Programmer have to define

1. Entity class
2. Repository interface

CRUD : Create Read Update and Delete

- We can perform basic operations like SELECT, INSERT, UPDATE and DELET using CrudRepository.

1. **save(obj):**

This method behaves like either INSERT(persist) or UPDATE(merge) operation based on PrimaryKey given inside object.

2. **saveAll(Iterable itbl):**

- This method is used to insert multiple rows at a time

3. **findAll():Iterable<T>**

- This method is used to fetch all rows from DB table.
- Returns data as Iterable<T>.

4. **count():int**

- This method is used to fetch no.of rows present in DB table

5. **existsById(id):boolean**

- This method is used to find given id exist in DB or not?

CrudRepository<T,ID>:

findById(ID id) : Optional<T>

- This method is used to fetch one row data from database using PrimaryKey value.

Optional<T> (JDK 1.8)

- `isPresent()` : check data is not null or not?
- `isEmpty()` [JDK 11] : check data is null or not?
- `get()` : returns actual object

deleteById(id) : This method is used to remove one row data from DB based on Primary Key value.

deleteAll(): This method removes all rows data one by one row.

delete(object): This method is used to remove one row data based on object reference.

PagingAndSortingRepository<T,ID>

This Repository is used to fetch data only using sorting concepts (ASC/DESC) or using Pagination concept (Fetch Data Page by Page).

- This Repository internally extends CrudRepository.
- It has 2 methods to fetch data only
 1. findAll(Sort): Iterable<T>
 2. findAll(Pageable pageable): Page<T>
- SQL: Database supports sorting data based on column using order by clause with ASC(default) and DESC

Pagination

- It is a process of fetching data page by page (With some limited size)
- `findAll()` is a method that gets all rows from DB table.

ex: `select * from emptab;`

- If we fetch all rows and display at browser, incase of large data it will get crashed. ex: 200K Rows.
- Instead of fetching all rows at a time use pagination. Fetch data using some limit ex: 20 rows at a time.

- PagingAndSortingRepository has given findAll() method to fetch data using pagination.

findAll(Pageable) : Page<T>

- Here input is Pageable which means = Page Number + Page Size
- PageNumber starts from zero. PageSize should be atleast one.

Content is List<T>

- Limit is the concept used to fetch data based on Row index and max rows
select * from table limit rowIndexToStart, MaxRowstoFetch;

JpaRepository

➤ Supports ALL JPA Operations,

1. Collections Mapping (Collections with Primitives)

List<String>, Set<Integer>

2. Association Mapping (1...1/1...*/...1/*....*)

3. Custom Query Language (SELECT/NON-SELECT -- UPDATE and DELETE)

Collections Mapping (Collections with Primitives)

- If we create a collection variable in Entity class then it creates a child table (Primary Key-Foreign Key link) in Database.
- Every Child table is connected with Parent table using FK Column.
- Supported collections are: List, Set, Map

Index Based Collections	Non-Index Based Collections
List, Map	Set

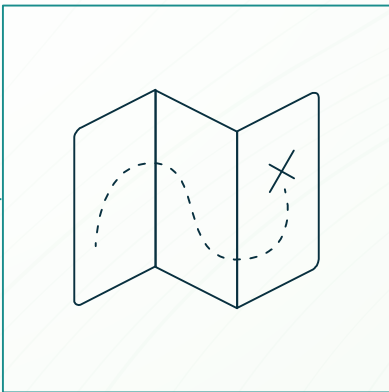
- For every collection variable, we need to apply `@ElementCollection` inside Entity.

Custom Query : @Query

- Pre-defined methods from Repository interfaces to perform database operations are : save(), findAll(), findById(), delete()..etc
- If want to define our own query(JPQL-HQL/SQL) to execute at DB.
- @Query Annotation is used for defining custom queries in Spring Data JPA.
- @Query annotation can execute (SELECT, UPDATE and DELETE).
- SQL is database Dependent.

JPQL/HQL:

- JPQL - HQL (Java Persistence Query Language - Hibernate Query Language)
- It is a query language, it is used to define queries (SELECT, UPDATE and DELETE)
- JPQL-HQL is a database independent.
- In Query, Java words like className, variableName are case-sensitive
- SQL words : select, from, where, count, group by, order by... are case-insensitive



Thanks!

Any questions?

You can find me at sayeedunnisa.js@gmail.com